
Control Plane (Expanded)

1. **kube-apiserver** – Central API hub that validates and processes REST requests.
2. **etcd** – Distributed key-value store holding the cluster's state and configuration.
3. **kube-scheduler** – Assigns Pods to Nodes based on constraints and available resources.
4. **kube-controller-manager** – Manages core controllers like Node, Job, and ReplicaSet.
5. **cloud-controller-manager** – Integrates Kubernetes with underlying cloud infrastructure.
6. **Custom Resource Definitions (CRD)** – Extends Kubernetes API with custom resources.
7. **Admission Controllers** – Pluggable components that validate or mutate API requests.
8. **API Aggregation Layer** – Allows extending Kubernetes with new APIs under `/apis`.

Node-Level (Expanded)

9. **kubelet** – Agent on each node that communicates with the API server and manages pods.
10. **kube-proxy** – Manages network routing for services and pods on each node.
11. **CRI (Container Runtime Interface)** – Standard for connecting container runtimes to kubelet.
12. **Containerd / CRI-O** – Popular CRI-compliant runtimes used to run containers.
13. **Node Problem Detector** – Monitors node-level issues and reports them to the control plane.
14. **CSI Node Plugin** – Handles node-specific volume lifecycle for persistent storage.

Workloads

15. **Pod** – Basic unit of deployment that holds one or more containers.
16. **ReplicaSet** – Ensures a specific number of pod replicas are running.
17. **Deployment** – Declarative updates and rollbacks for stateless apps.
18. **StatefulSet** – Manages stateful pods with stable network identities.
19. **DaemonSet** – Ensures a pod runs on all (or selected) nodes.
20. **Job** – Executes a pod until completion for batch/one-off tasks.
21. **CronJob** – Schedules jobs to run periodically using Cron syntax.

Networking

22. **Service** – Exposes a stable endpoint for accessing one or more pods.

- 23. **Endpoints** – Maps a Service to actual Pod IPs.
 - 24. **Ingress** – Defines HTTP(S) routing rules to services.
 - 25. **Ingress Controller** – Implements the Ingress API (e.g., NGINX, Traefik).
 - 26. **NetworkPolicy** – Controls allowed communication between pods.
 - 27. **CNI Plugins** – Network implementations for pod networking (e.g., Calico, Flannel, Cilium).
 - 28. **Multus CNI** – Allows attaching multiple network interfaces to pods.
 - 29. **Service Mesh (e.g., Istio/Linkerd)** – Adds observability, traffic control, and security across services.
-

Storage & Volumes

- 30. **PersistentVolume (PV)** – Abstraction for cluster-wide storage resources.
 - 31. **PersistentVolumeClaim (PVC)** – A request for PV storage by a user or pod.
 - 32. **StorageClass** – Defines dynamic provisioning rules for volumes.
 - 33. **VolumeSnapshot** – Enables backups by capturing point-in-time snapshots of volumes.
 - 34. **CSI (Container Storage Interface)** – Pluggable storage driver framework.
 - 35. **VolumeAttachment** – Links storage volumes to specific nodes.
-

Security

- 36. **Secret** – Stores sensitive data like passwords or tokens.
 - 37. **ConfigMap** – Stores configuration data in key-value format.
 - 38. **PodSecurityContext** – Sets security parameters for pods and containers.
 - 39. **Pod Security Admission (PSA)** – Enforces pod security standards (`restricted`, `baseline`).
 - 40. **RBAC (Role-Based Access Control)** – Manages user and service permissions.
 - 41. **Role / ClusterRole** – Defines allowed actions on resources.
 - 42. **RoleBinding / ClusterRoleBinding** – Assigns roles to users or service accounts.
 - 43. **Security Policies (OPA/Gatekeeper)** – Applies advanced policies using Open Policy Agent.
 - 44. **NetworkPolicy Controller** – Enforces traffic rules defined in NetworkPolicy.
-

Extensibility & Customization

- 45. **CRDs (Custom Resource Definitions)** – Extend Kubernetes with custom resource types.
- 46. **Operators** – Custom controllers that automate complex app lifecycle management.
- 47. **Admission Webhooks** – Custom logic to validate or mutate requests to the API.
- 48. **MutatingWebhookConfiguration** – Registers mutating admission webhooks.
- 49. **ValidatingWebhookConfiguration** – Registers validating admission webhooks.

50. **APIService** – Registers external APIs with Kubernetes using the Aggregation Layer.



Observability & Autoscaling

- 51. **Metrics Server** – Collects resource metrics for nodes and pods.
 - 52. **Horizontal Pod Autoscaler (HPA)** – Scales pods based on resource usage.
 - 53. **Vertical Pod Autoscaler (VPA)** – Automatically adjusts resource requests/limits.
 - 54. **Cluster Autoscaler** – Adds/removes nodes from the cluster based on demand.
 - 55. **Kube-state-metrics** – Exposes cluster state to Prometheus.
 - 56. **Event Exporter** – Forwards Kubernetes events to external systems.
 - 57. **Prometheus Operator** – Manages Prometheus monitoring stack.
 - 58. **Alertmanager** – Handles alerts sent by Prometheus.
 - 59. **Grafana** – Dashboards for visualizing Kubernetes metrics.
 - 60. **OpenTelemetry Collector** – Gathers distributed traces, logs, and metrics.
-



Lifecycle & Scheduling Tools

- 61. **Init Containers** – Containers that run before main app containers.
 - 62. **Ephemeral Containers** – Temporary containers for debugging running pods.
 - 63. **Affinity & Anti-Affinity Rules** – Influence pod scheduling decisions.
 - 64. **Taints and Tolerations** – Prevent or allow pods to be scheduled on specific nodes.
 - 65. **PriorityClass** – Influences pod preemption and scheduling order.
 - 66. **PodDisruptionBudget (PDB)** – Controls voluntary pod disruptions during upgrades.
 - 67. **PodPreset (deprecated)** – Used to inject settings into pods at creation.
-



Namespaces & Multi-Tenancy

- 68. **Namespace** – Logical partitioning of Kubernetes resources.
 - 69. **ResourceQuota** – Limits the amount of resources a namespace can use.
 - 70. **LimitRange** – Sets default resource limits and requests per container.
-



Cluster Management Tools

- 71. **kubeadm** – CLI tool for bootstrapping a Kubernetes cluster.
- 72. **kops** – Tool for managing Kubernetes clusters on AWS.
- 73. **kind** – Run local Kubernetes clusters using Docker containers.
- 74. **minikube** – Run Kubernetes locally for testing and development.
- 75. **kubectrl** – Command-line tool for interacting with Kubernetes clusters.
- 76. **Helm** – Package manager for Kubernetes applications (via charts).
- 77. **Argo CD** – Declarative GitOps continuous delivery tool for Kubernetes.

78. **Flux** – GitOps operator that synchronizes Git with your cluster state.

Authentication & Access

79. **ServiceAccount** – Identity for processes in pods to interact with the API.

80. **TokenReview API** – Verifies bearer tokens for authentication.

81. **Webhook Token Authentication** – Custom auth mechanism via external service.

82. **OpenID Connect (OIDC)** – Integration with external identity providers.

If you'd like, I can turn this into:

-  A downloadable PDF
-  A visual mind map
-  A cheat sheet poster

Let me know your preferred format or if you want an even deeper layer (e.g., SIG-level architecture or plugin architecture).